Operating Systems, Section A
Homework 4: Exploring Compilation Units in C Using nm, objdump, and readelf

Ani Hambardzumyan

Source code: https://github.com/anii-h/os_hw_4/tree/main

math_utils.h:
The header file contains the signature of the square functions, giving the information that it accepts an integer parameter, and returns an integer. It allows the main.c to know that the function exists.

math_utils.c:
Contains the actual implementation of square() function.

main.c:
Initializes n to 4, and calls square(n). Prints both results in the standard output.

First, the program was compiled into obj files without linking:
gcc -c main.c
gcc -c math_utils.c

-c flag stops the compilation and produces -o obj file instead of an executable file
So, in result, we got two object files: main.o math_utils.o

Then I linked the two obj files together getting a final executable.
gcc main.o math_utils.o -o square_prog

nm:
Output for main.o:
0000000000000000 T main
                 U printf
                 U square

Symbols - main, printf, square
Main is of type T(text segment)
Printf and square are undefined

For math_utils.o:
0000000000000000 T square
We only have the symbol square in the text segment. (does not depend on another, user-defined function)

For final executable square_prog:
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$ nm square_prog
0000000000003dc8 d _DYNAMIC
0000000000003fb8 d _GLOBAL_OFFSET_TABLE_
0000000000002000 R _IO_stdin_used
         w _ITM_deregisterTMCloneTable
         w _ITM_registerTMCloneTable
0000000000002118 r __FRAME_END__
0000000000002014 r __GNU_EH_FRAME_HDR
0000000000004010 D __TMC_END__
000000000000038c r __abi_tag
0000000000004010 B __bss_start
         w __cxa_finalize@GLIBC_2.2.5
0000000000004000 D __data_start
0000000000001100 t __do_global_dtors_aux
0000000000003dc0 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003db8 d __frame_dummy_init_array_entry
         w __gmon_start__
         U __libc_start_main@GLIBC_2.34
0000000000004010 D _edata
0000000000004018 B _end
000000000000119c T _fini
0000000000001000 T _init
0000000000001060 T _start
0000000000004010 b completed.0
0000000000004000 W data_start
0000000000001090 t deregister_tm_clones
0000000000001140 t frame_dummy
0000000000001149 T main
         U printf@GLIBC_2.2.5
00000000000010c0 t register_tm_clones
0000000000001188 T square

We have a lot of important symbols, but
0000000000001149 T main U printf@GLIBC_2.2.5 0000000000001188 T square
The square is no longer undefined, compared to the output of nm main.o.
And also the program does not start at 00000000000000. Printf is still undefined since it is a libc function and is called during the runtime.

objdump:

For main.o:
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$ objdump -d main.o

main.o: file format elf64-x86-64

Disassembly of section .text:

```
0000000000000000 <main>:
   0:   f3 0f 1e fa             endbr64
   4:   55                      push   %rbp
   5:   48 89 e5                mov    %rsp,%rbp
   8:   48 83 ec 10             sub    $0x10,%rsp
   c:   c7 45 fc 04 00 00 00          movl   $0x4,-0x4(%rbp)
  13:   8b 45 fc                mov    -0x4(%rbp),%eax
  16:   89 c7                   mov    %eax,%edi
  18:   e8 00 00 00 00          call   1d <main+0x1d>
  1d:   89 c2                   mov    %eax,%edx
  1f:   8b 45 fc                mov    -0x4(%rbp),%eax
  22:   89 c6                   mov    %eax,%esi
  24:   48 8d 05 00 00 00 00          lea    0x0(%rip),%rax        # 2b <main+0x2b>
  2b:   48 89 c7                mov    %rax,%rdi
  2e:   b8 00 00 00 00          mov    $0x0,%eax
  33:   e8 00 00 00 00          call   38 <main+0x38>
  38:   b8 01 00 00 00          mov    $0x1,%eax
  3d:   c9                      leave
  3e:   c3                      ret
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$
```

In the beginning we can notice two calls:

```
0000000000000000 <main>:
   ...
   18: e8 00 00 00 00        call   1d <main+0x1d>
   ...
   24: 48 8d 05 00 00 00 00  lea ...
   ...
   33: e8 00 00 00 00        call   38 <main+0x38>
```

The first one is a call to square(n)
The second one is a call to printf.

For math_utils.o:

anih@ubuntu-aua-2026:~/homeworks/os_hw_4$ objdump -d math_utils.o

math_utils.o: file format elf64-x86-64

Disassembly of section .text:

```
0000000000000000 <square>:
   0:    f3 0f 1e fa             endbr64
   4:    55                      push   %rbp
   5:    48 89 e5                mov    %rsp,%rbp
   8:    89 7d fc                mov    %edi,-0x4(%rbp)
   b:    8b 45 fc                mov    -0x4(%rbp),%eax
   e:    0f af c0                imul   %eax,%eax
  11:    5d                      pop    %rbp
  12:    c3                      ret
```

anih@ubuntu-aua-2026:~/homeworks/os_hw_4$

Here imul %eax,%eax this line is the n * n, as we can guess.

For square_prog:
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$ objdump -d square_prog

square_prog: file format elf64-x86-64

```
Disassembly of section .init:

0000000000001000 <_init>:
   1000:	f3 0f 1e fa 	endbr64
   1004:	48 83 ec 08 	sub    $0x8,%rsp
   1008:	48 8b 05 d9 2f 00 00 	mov    0x2fd9(%rip),%rax        # 3fe8
<__gmon_start__@Base>
   100f:	48 85 c0 	test   %rax,%rax
   1012:	74 02 	je     1016 <_init+0x16>
   1014:	ff d0 	call   *%rax
   1016:	48 83 c4 08 	add    $0x8,%rsp
   101a:	c3 	ret

Disassembly of section .plt:

0000000000001020 <.plt>:
   1020:	ff 35 9a 2f 00 00 	push   0x2f9a(%rip)        # 3fc0
<_GLOBAL_OFFSET_TABLE_+0x8>
   1026:	ff 25 9c 2f 00 00 	jmp    *0x2f9c(%rip)        # 3fc8
<_GLOBAL_OFFSET_TABLE_+0x10>
   102c:	0f 1f 40 00 	nopl   0x0(%rax)
   1030:	f3 0f 1e fa 	endbr64
   1034:	68 00 00 00 00 	push   $0x0
   1039:	e9 e2 ff ff ff 	jmp    1020 <_init+0x20>
   103e:	66 90 	xchg   %ax,%ax

Disassembly of section .plt.got:

0000000000001040 <__cxa_finalize@plt>:
   1040:	f3 0f 1e fa 	endbr64
   1044:	ff 25 ae 2f 00 00 	jmp    *0x2fae(%rip)        # 3ff8
<__cxa_finalize@GLIBC_2.2.5>
   104a:	66 0f 1f 44 00 00 	nopw   0x0(%rax,%rax,1)

Disassembly of section .plt.sec:

0000000000001050 <printf@plt>:
   1050:	f3 0f 1e fa 	endbr64
```

```
   1054:	ff 25 76 2f 00 00	jmp    *0x2f76(%rip)        # 3fd0
<printf@GLIBC_2.2.5>
   105a:	66 0f 1f 44 00 00	nopw   0x0(%rax,%rax,1)

Disassembly of section .text:

0000000000001060 <_start>:
   1060:	f3 0f 1e fa		endbr64
   1064:	31 ed			xor    %ebp,%ebp
   1066:	49 89 d1		mov    %rdx,%r9
   1069:	5e			pop    %rsi
   106a:	48 89 e2		mov    %rsp,%rdx
   106d:	48 83 e4 f0		and    $0xfffffffffffffff0,%rsp
   1071:	50			push   %rax
   1072:	54			push   %rsp
   1073:	45 31 c0		xor    %r8d,%r8d
   1076:	31 c9			xor    %ecx,%ecx
   1078:	48 8d 3d ca 00 00 00	lea    0xca(%rip),%rdi        # 1149 <main>
   107f:	ff 15 53 2f 00 00	call   *0x2f53(%rip)        # 3fd8
<__libc_start_main@GLIBC_2.34>
   1085:	f4			hlt
   1086:	66 2e 0f 1f 84 00 00	cs nopw 0x0(%rax,%rax,1)
   108d:	00 00 00

0000000000001090 <deregister_tm_clones>:
   1090:	48 8d 3d 79 2f 00 00	lea    0x2f79(%rip),%rdi        # 4010
<__TMC_END__>
   1097:	48 8d 05 72 2f 00 00	lea    0x2f72(%rip),%rax        # 4010
<__TMC_END__>
   109e:	48 39 f8		cmp    %rdi,%rax
   10a1:	74 15			je     10b8 <deregister_tm_clones+0x28>
   10a3:	48 8b 05 36 2f 00 00	mov    0x2f36(%rip),%rax        # 3fe0
<_ITM_deregisterTMCloneTable@Base>
   10aa:	48 85 c0		test   %rax,%rax
   10ad:	74 09			je     10b8 <deregister_tm_clones+0x28>
   10af:	ff e0			jmp    *%rax
   10b1:	0f 1f 80 00 00 00 00	nopl   0x0(%rax)
   10b8:	c3			ret
   10b9:	0f 1f 80 00 00 00 00	nopl   0x0(%rax)
```

```
00000000000010c0 <register_tm_clones>:
   10c0:	48 8d 3d 49 2f 00 00	lea    0x2f49(%rip),%rdi        # 4010
<__TMC_END__>
   10c7:	48 8d 35 42 2f 00 00	lea    0x2f42(%rip),%rsi        # 4010
<__TMC_END__>
   10ce:	48 29 fe		sub    %rdi,%rsi
   10d1:	48 89 f0		mov    %rsi,%rax
   10d4:	48 c1 ee 3f		shr    $0x3f,%rsi
   10d8:	48 c1 f8 03		sar    $0x3,%rax
   10dc:	48 01 c6		add    %rax,%rsi
   10df:	48 d1 fe		sar    $1,%rsi
   10e2:	74 14			je     10f8 <register_tm_clones+0x38>
   10e4:	48 8b 05 05 2f 00 00	mov    0x2f05(%rip),%rax        # 3ff0
<_ITM_registerTMCloneTable@Base>
   10eb:	48 85 c0		test   %rax,%rax
   10ee:	74 08			je     10f8 <register_tm_clones+0x38>
   10f0:	ff e0			jmp    *%rax
   10f2:	66 0f 1f 44 00 00	nopw   0x0(%rax,%rax,1)
   10f8:	c3			ret
   10f9:	0f 1f 80 00 00 00 00	nopl   0x0(%rax)

0000000000001100 <__do_global_dtors_aux>:
   1100:	f3 0f 1e fa		endbr64
   1104:	80 3d 05 2f 00 00 00	cmpb   $0x0,0x2f05(%rip)        # 4010
<__TMC_END__>
   110b:	75 2b			jne    1138 <__do_global_dtors_aux+0x38>
   110d:	55			push   %rbp
   110e:	48 83 3d e2 2e 00 00	cmpq   $0x0,0x2ee2(%rip)        # 3ff8
<__cxa_finalize@GLIBC_2.2.5>
   1115:	00
   1116:	48 89 e5		mov    %rsp,%rbp
   1119:	74 0c			je     1127 <__do_global_dtors_aux+0x27>
   111b:	48 8b 3d e6 2e 00 00	mov    0x2ee6(%rip),%rdi        # 4008
<__dso_handle>
   1122:	e8 19 ff ff ff		call   1040 <__cxa_finalize@plt>
   1127:	e8 64 ff ff ff		call   1090 <deregister_tm_clones>
   112c:	c6 05 dd 2e 00 00 01	movb   $0x1,0x2edd(%rip)        # 4010
<__TMC_END__>
   1133:	5d			pop    %rbp
   1134:	c3			ret
```

```
    1135:       0f 1f 00                nopl   (%rax)
    1138:       c3                      ret
    1139:       0f 1f 80 00 00 00 00            nopl   0x0(%rax)

0000000000001140 <frame_dummy>:
    1140:       f3 0f 1e fa             endbr64
    1144:       e9 77 ff ff ff          jmp    10c0 <register_tm_clones>

0000000000001149 <main>:
    1149:       f3 0f 1e fa             endbr64
    114d:       55                      push   %rbp
    114e:       48 89 e5                mov    %rsp,%rbp
    1151:       48 83 ec 10             sub    $0x10,%rsp
    1155:       c7 45 fc 04 00 00 00            movl   $0x4,-0x4(%rbp)
    115c:       8b 45 fc                mov    -0x4(%rbp),%eax
    115f:       89 c7                   mov    %eax,%edi
    1161:       e8 22 00 00 00          call   1188 <square>
    1166:       89 c2                   mov    %eax,%edx
    1168:       8b 45 fc                mov    -0x4(%rbp),%eax
    116b:       89 c6                   mov    %eax,%esi
    116d:       48 8d 05 90 0e 00 00            lea    0xe90(%rip),%rax        # 2004
<_IO_stdin_used+0x4>
    1174:       48 89 c7                mov    %rax,%rdi
    1177:       b8 00 00 00 00          mov    $0x0,%eax
    117c:       e8 cf fe ff ff          call   1050 <printf@plt>
    1181:       b8 01 00 00 00          mov    $0x1,%eax
    1186:       c9                      leave
    1187:       c3                      ret

0000000000001188 <square>:
    1188:       f3 0f 1e fa             endbr64
    118c:       55                      push   %rbp
    118d:       48 89 e5                mov    %rsp,%rbp
    1190:       89 7d fc                mov    %edi,-0x4(%rbp)
    1193:       8b 45 fc                mov    -0x4(%rbp),%eax
    1196:       0f af c0                imul   %eax,%eax
    1199:       5d                      pop    %rbp
    119a:       c3                      ret

Disassembly of section .fini:
```

```
000000000000119c <_fini>:
    119c:       f3 0f 1e fa             endbr64
    11a0:       48 83 ec 08             sub    $0x8,%rsp
    11a4:       48 83 c4 08             add    $0x8,%rsp
    11a8:       c3                      ret
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$
```

This is the disassembly of our final executable, which is significantly longer.
0000000000001149 <main>
0000000000001188 <square>

Readelf:

main.o

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          696 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
  Size of section headers:           64 (bytes)
  Number of section headers:         14
  Section header string table index: 13
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$
```

math_utils.o

```
ELF Header:
```

```
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          472 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
  Size of section headers:           64 (bytes)
  Number of section headers:         12
  Section header string table index: 11
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$
```

square_prog:

```
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$ readelf -h square_prog
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1060
  Start of program headers:          64 (bytes into file)
  Start of section headers:          14048 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
```

```
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
anih@ubuntu-aua-2026:~/homeworks/os_hw_4$
```

For the mian.o(and math_utils) we can see the line: Type: REL (Relocatable file) - which means that it is not an executable file yet, but relocatable(can be rrelocated).
Also, both math_utils.o and main.o have no program headers.

```
Class:        ELF64
Data:         2's complement, little endian
Type:         DYN (Position-Independent Executable file)
Machine:      Advanced Micro Devices X86-64
Entry point:  0x1060
```

However, the executable file has a real entry point, and is DYN (Position-Independent Executable file (pie file)). It also has program headers.

main.o constants 14 sections:
importantly::
.text
.rela.text
.data
.bss
.rodata
.comment
.note.GNU-stack
.note.gnu.property
.eh_frame
.rela.eh_frame
.symtab .strtab .shstrtab

math_utils.o - 12 sections:
.text
.data
.bss

.comment
.note.GNU-stack
.note.gnu.property
.eh_frame
.rela.eh_frame
.symtab
.strtab
.shstrtab

Comparing to the main.o's sections we can see that math_utils.o does not have .rela.text section, which contains relocation information associated with the text section, and since main calls 2 functions whose final addresses are not known, it has this section. But math_utils.c doesnt use any function, so the .rela.text is not needed.

Square_prog - 31 sections:
(importantly)
.interp
.gnu.hash
.dynsym
.dynstr
.gnu.version
.rela.dyn
.rela.plt
.init
.plt
.plt.got
.plt.sec
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.got
.data
.bss
.symtab

.strtab
.shstrtab

The role of the linker:

The linker is responsible for combining separately compiled object files into a single executable.
main.o contains a reference to square(), but does not contain its implementation. math_utils.o contains the implementation and the linker resolves this relationship and places both pieces of code into the final executable. It also performs relocation, meaning it updates references in the machine code so that calls and addresses point to the correct locations.